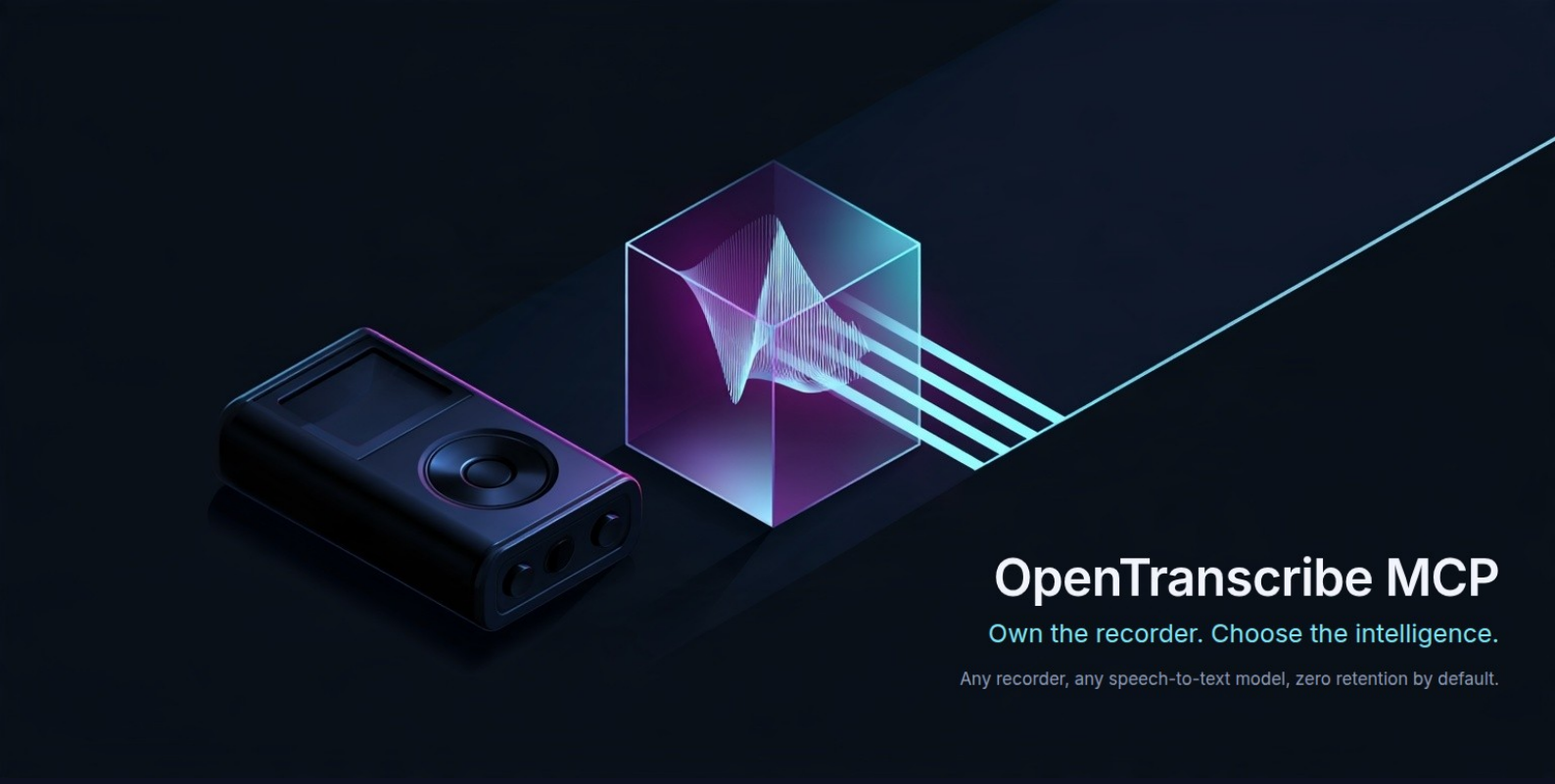

OPENTRANSCRIBE MCP

Transcribe your Plaud recordings on Ubuntu

A step-by-step guide for people who do not write code

Apache-2.0 · github.com/fbossiere/open-transcribe-mcp

Your Plaud recorder captures the meeting. This guide shows you how to turn those recordings into written text on your own Ubuntu computer, using the transcription service of your choice rather than the one that came with the device.

You do not need to be a developer. You will copy and paste a handful of commands into a black window called the **Terminal**, and answer a few questions along the way. Set aside about **thirty minutes** the first time. Afterwards, transcribing a new recording takes one command and a couple of minutes.

> What you are actually installing
>
> OpenTranscribe is a small program that runs on your computer. It takes an audio file, sends it to a speech-to-text service you have chosen, and hands you back a clean transcript. It does not keep your audio, and it does not send anything anywhere except to the one service you configured.

How the pieces fit together

There are four things involved, and it helps to picture them before you start.

The piece	What it does	Where it lives
Your Plaud recorder	Records the conversation	In your pocket
The Plaud app or website	Holds your recordings; lets you download the audio	Your phone and Plaud's servers
OpenTranscribe	Receives an audio file and returns a transcript	Your Ubuntu computer
A speech-to-text provider	Does the actual listening and writing	The provider's servers

The audio makes exactly one journey off your computer: from OpenTranscribe to the provider you picked. Nothing else is sent anywhere, and no transcript is stored unless you deliberately turn storage on.

Before you start

You will need:

- **An Ubuntu computer.** Ubuntu 22.04 or 24.04 both work. Anything reasonably recent is fine; transcription happens on the provider's servers, not on your machine, so an old laptop is perfectly adequate.
- **Your Plaud device and the Plaud app**, signed in to your account.
- **An internet connection.**
- **A payment card**, to open an account with a speech-to-text provider. These services charge per hour of audio; a one-hour meeting typically costs a few tens of cents. You are not signing up for a subscription.
- **About thirty minutes.**

> Before you record anyone
>
> Recording and transcribing people is regulated in most countries. Make sure you have the right to record the conversation and to process it, and tell people when they are being recorded. This guide assumes the recordings are yours to use.

Step 1 — Open the Terminal

Press the **Super key** (the one with the Windows or Ubuntu logo), type `terminal`, and press **Enter**. A window with a text prompt opens. This is where every command in this guide goes.

To run a command: select it here, copy it, click inside the Terminal window, paste it with **Ctrl + Shift + V**, and press **Enter**. Then wait for the prompt to come back before running the next one.

> Nothing appears when you type your password
>
> When Ubuntu asks for your password, the Terminal shows nothing at all — no dots, no stars. That is normal. Type it and press Enter.

Step 2 — Install the two tools you need

The first command installs `git`, which downloads software from the internet. The second installs `uv`, which sets up the Python pieces OpenTranscribe needs.

```
sudo apt update && sudo apt install -y git curl
```

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Close the Terminal window and open a new one, so it picks up the newly installed `uv`. Check that both tools are there:

```
git --version && uv --version
```

You should see two version numbers. If you see `command not found`, close and reopen the Terminal once more.

Step 3 — Download OpenTranscribe

```
git clone https://github.com/fbossiere/open-transcribe-mcp.git ~/open-transcribe
```

```
cd ~/open-transcribe && uv sync
```

The second command takes a minute or two and prints a long list of package names. That is it installing everything OpenTranscribe depends on. When the prompt comes back, you are done.

> About that `cd`
>
> `cd` means "change directory" — it moves the Terminal into the folder you just downloaded. Every command from here on assumes you are inside `~/open-transcribe`. If you open a new Terminal window later, run `cd ~/open-transcribe` first.

Step 4 — Choose a transcription provider and get a key

OpenTranscribe can talk to three providers. You only need one. They differ in price and in what they can do:

Provider	Tells speakers apart	Per hour of audio	Sign-up effort
ElevenLabs Scribe v2	Yes	about $0.22	One page: create an account, copy a key
Microsoft MAI-Transcribe-2	Yes	about $0.10	Longer: you must create an Azure resource first
Groq Whisper	No	$0.04 to $0.11	One page

For meetings, choose ElevenLabs. It labels who said what — which is the whole point of a meeting transcript — and getting a key takes two minutes. Microsoft is half the price and just as capable, but it makes you create a cloud resource first, which is a detour if this is your first time. Groq is cheapest and fastest and returns one undifferentiated block of text: fine for a voice memo, frustrating for a four-person meeting.

> About those prices
>
> They come from the pricing table shipped with the project and are there to give you a sense of scale, not a quote. Providers change their prices; check theirs before you commit to one.

To get an ElevenLabs key:

1. Go to elevenlabs.io and create an account.
2. Open your profile menu and find the **API keys** section.
3. Create a new key and copy it. It starts with `sk_`.
4. Paste it somewhere safe for a moment — you need it in the next step.

> Treat the key like a password
>
> Anyone holding that key can spend money on your account. Do not email it, do not put it in a document you share, and do not paste it into a chat.

Step 5 — Write your settings file

OpenTranscribe reads its settings from a file called `.env`. Create it with this command — but **replace the two placeholder values first**:

- Replace `sk_PASTE_YOUR_ELEVENLABS_KEY_HERE` with the key from Step 4.
- Replace `pick-a-long-random-password-here` with any long random phrase you invent. It is a local password that stops other programs on your computer from using your transcription budget. Twenty characters of nonsense is ideal.

```
cat > ~/open-transcribe/.env <<'EOF'
OT_DEFAULT_PROVIDER=elevenlabs
OT_DEFAULT_MODEL=scribe-v2
OT_ELEVENLABS__API_KEY=sk_PASTE_YOUR_ELEVENLABS_KEY_HERE
OT_ELEVENLABS__ZERO_RETENTION=true

OT_SECURITY__AUTH_MODE=bearer
OT_SECURITY__BEARER_TOKEN=pick-a-long-random-password-here

OT_SECURITY__REQUIRE_HTTPS_SOURCES=false
OT_SECURITY__ALLOW_PRIVATE_URLS=true
EOF
```

Two lines deserve an explanation.

`OT_ELEVENLABS__ZERO_RETENTION=true` asks ElevenLabs not to keep a copy of your audio. Leave it on.

The last two lines are the ones that let OpenTranscribe read files from your own computer. Out of the box it only accepts audio published on the public internet, as a safety measure for servers exposed online. Your installation is not exposed online — it listens only to your own machine — so relaxing that restriction is safe here.

> One condition
>
> Those last two lines are safe **only** because this server runs on your own computer and is not reachable from the internet. If you ever put OpenTranscribe on a shared machine or a public server, delete both lines first.

> Why not copy `.env.example`?
>
> The repository ships an example settings file, but several of its lines are left deliberately blank, and the server refuses to start when it finds a blank value where it expects a number or an address. The block above is complete and starts cleanly. Use it as written.

Step 6 — Start the service

```
cd ~/open-transcribe && uv run open-transcribe-mcp
```

After a few seconds you will see a line ending in:

```
Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Leave this window open. The service runs for as long as this window is running. Closing it stops the service.

Now open a **second** Terminal window (**Ctrl + Alt + T**) and check that the service is healthy:

```
curl http://localhost:8000/readyz
```

You should see:

```
{"status":"ready","configured_providers":["elevenlabs"]}
```

If `configured_providers` is empty, your key did not load. Go back to Step 5 and check for a typo or a stray space.

Step 7 — Try it on the sample recording

Before involving your own audio, prove the whole chain works. The project ships a short bilingual test recording with two voices.

In the second Terminal window, run this — replacing the password with the one you invented in Step 5:

```
cd ~/open-transcribe && uv run python examples/transcribe.py --token pick-a-long-random-password-here
```

After a few seconds you will see a block of structured text containing the transcript, which speaker said each line, and what the request cost. If you got that, everything is working, and the rest is easy.

Step 8 — Get the recording off your Plaud device

This is the one step that happens outside the Terminal.

The reliable way: export from Plaud

Plaud devices upload their recordings to the Plaud app over Bluetooth, and from there to your Plaud account. Download the audio from the app or from the web player:

1. Open the **Plaud app** on your phone, or go to Plaud's web player in a browser on your Ubuntu machine, and sign in.
2. Open the recording you want.
3. Use the **Share** or **Export** control — usually an icon in the top-right corner.
4. Choose to export the **audio** (as opposed to the summary or the transcript). The web player exports MP3; the phone app can also export WAV. Either is fine.
5. Save the file into your **Downloads** folder on the Ubuntu machine. If you exported from your phone, send it to yourself by email, or use Plaud's web player on the Ubuntu machine directly, which is simpler.

> Do this once, then keep the file
>
> Plaud's export links expire. Download the audio to your own disk rather than trying to pass a link to OpenTranscribe — a downloaded file will still work tomorrow.

The legacy way: plugging the device in

Older Plaud Note units, on firmware versions before V2.1, could be plugged into a computer with the USB cable and browsed like a memory stick, with recordings in folders named `NOTES` and `CALLS`. Plaud removed that capability in firmware V2.1, and the newer NotePin and Note Pro models never had it. If your device shows up in the Files application with those folders, you can copy the audio directly. If it does not, use the export route above — it is not a fault with your cable.

> Stay on the marked path
>
> Use the export features Plaud gives you. Do not try to work around the device's protections or extract credentials from the app. Aside from being against Plaud's terms, it is not necessary: the export button gives you the same audio.

Step 9 — Transcribe your recording

Suppose your file is `~/Downloads/team-meeting.mp3`. In the second Terminal window, run:

```
cd ~/open-transcribe && uv run python examples/plaud_transcribe.py ~/Downloads/team-meeting.mp3 --token pick-a-long-random-password-here
```

Replace the file path with your own, and the password with yours. You will see:

```
Transcribing team-meeting.mp3 (18.4 MB)…
Provider: elevenlabs / scribe-v2
Languages: eng
Saved: /home/you/Downloads/team-meeting.txt
```

The transcript is written next to your audio file, with the same name and a `.txt` ending. Open it by double-clicking it in the Files application. It looks like this:

```
[00:00] SPEAKER_01
Right, shall we start? Everyone got the agenda?

[00:07] SPEAKER_02
Yes. I had one question about the second item.
```

The timestamps are minutes and seconds from the start of the recording, so you can jump straight to a passage in the audio. `SPEAKER_01` and `SPEAKER_02` are the service's way of saying "this is a different voice" — it does not know anyone's name, and you can rename them yourself in the text file.

Useful variations

Add these to the end of the command when you need them:

What you want	Add this
Tell it the language, if detection struggles	`--language fr`
Tell it how many people are speaking	`--speakers 4`
One block of text, no speaker labels	`--no-diarization`
Save the transcript somewhere specific	`--out ~/Documents/meeting.txt`
Prefer the cheapest provider over the best	`--policy cost`

For example, a four-person French meeting:

```
uv run python examples/plaud_transcribe.py ~/Downloads/reunion.mp3 --token pick-a-long-random-password-here --language fr --speakers 4
```

Using it again tomorrow

The setup work is done for good. From now on, transcribing a recording is three things:

1. Open a Terminal, run `cd ~/open-transcribe && uv run open-transcribe-mcp`, and leave that window open.
2. Open a second Terminal and run the `plaud_transcribe.py` command with your file.
3. When you are finished for the day, click into the first window and press **Ctrl + C** to stop the service.

> Save yourself the typing
>
> Keep both long commands in a text file on your desktop and copy them from there.

When something goes wrong

What you see	What it means	What to do
`command not found: uv`	The Terminal has not noticed the new tool	Close the window, open a new one
`configured_providers":[]`	Your key was not read	Re-check Step 5 for typos and stray spaces
`PROVIDER_AUTHENTICATION_FAILED`	The provider rejected your key	The key is wrong, revoked, or the account has no credit. Create a fresh key
`SOURCE_URL_REJECTED`	The two local-access lines are missing	Re-check the last two lines of Step 5, then restart the service
`INVALID_AUDIO`	The file is not audio, or it is damaged	Play it first. Re-export it from Plaud
`SOURCE_TOO_LARGE`	Longer than the built-in limit	Add `OT_MAX_AUDIO_SIZE_MB=1000` to `.env` and restart, or split the recording
`Connection refused`	The service is not running	Go back to the first window; it should say `Uvicorn running`

What you see	What it means	What to do
It hangs for a long time	Long recordings simply take a while	A one-hour recording can take several minutes. Wait

To restart the service after changing `.env`: click into the first window, press **Ctrl + C**, then run the start command again.

What happens to your audio

Worth knowing, and worth being able to explain to the people you record:

- Your audio file stays on your disk. OpenTranscribe reads it, sends it to the one provider you configured, and deletes its working copy immediately.
- Nothing is logged: not the audio, not the transcript, not your key. The project sends no usage statistics anywhere.
- No transcript is kept by OpenTranscribe. The `.txt` file on your disk is the only copy it produces.
- With `OT_ELEVENLABS__ZERO_RETENTION=true`, ElevenLabs is asked not to keep a copy either. Their own terms govern what actually happens on their side.
- Plaud still holds whatever you uploaded to your Plaud account. This guide adds a second path for your audio; it does not remove the first one.

Where to go next

- **Change provider.** Edit `OT_DEFAULT_PROVIDER` in `.env` after adding that provider's key, and restart. Your commands do not change — that is the point of OpenTranscribe.
- **Connect it to an AI assistant.** OpenTranscribe speaks MCP, so an assistant that supports MCP can call it directly and work with your transcripts in conversation. See the project README.
- **Read the details.** The providers page explains what each service can and cannot do; the privacy page covers retention.

> Trademarks
>
> Plaud is a trademark of its respective owner. OpenTranscribe is an independent open-source project and is not affiliated with, endorsed by, or sponsored by Plaud, ElevenLabs, Microsoft, or Groq. Plaud's apps and export features change over time; if a menu has moved, the Plaud support site is the authority on where it went.